Coding & Solution

Date	15 July 2026
Team ID	xxxxxx
Project Name	Emotion Detection & Learning Support Engine
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of your implemented code and the overall solution, demonstrating working functionality, clean code practices, and alignment with defined software requirements.

Instructions:

1. Ensure your code is well-structured, commented, and follows best practices for your chosen technology stack.
2. The solution should address the core problem statement and implement the key functional requirements.
3. Include all source files, configuration files, and setup instructions needed to run the application.
4. Attach or link to your code repository (e.g., GitHub) in the table below.

Solution Summary

Field	Details
Repository Link / URL	https://github.com/Nisanthpakanati/EduEmotion-AI
Programming Language(s)	Python 3.12, HTML, CSS
Framework(s) Used	Streamlit, PyTorch, Hugging Face Transformers
Key Features Implemented	• Emotion Detection using BERT & BiLSTM • Mixed Emotion Prediction • Google Gemini AI Guidance • Analytics Dashboard (Plotly) • SQLite Database Storage • CSV Logging • Student Emotion History • Session Management
Pending / Incomplete Features	• Cloud Deployment • User Authentication with OAuth • Multi-user Support
Setup / Run Instructions	1. Clone the repository. 2. Install dependencies using pip install -r requirements.txt. 3. Configure the Gemini API key in the .env file. 4. Run the application using streamlit run app.py. 5. Open the local URL in the browser to access the application.

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Additional Notes / Comments:
The project follows a **modular architecture** with separate folders for authentication, analytics, database management, AI services, model training, utilities, and testing. Emotion detection is performed using **BERT** and **BiLSTM** models, while **Google Gemini API** generates personalized learning guidance. The application includes interactive visualizations, persistent data storage using SQLite and CSV files, and version control through GitHub.